

Quick Sort

- Quick Sort é um algoritmo de ordenação que organiza uma lista dividindo-a em partes menores.
- Ele escolhe um elemento chamado pivô, coloca os valores menores à esquerda e os maiores à direita
- Depois repete o mesmo processo em cada parte até que todos os elementos estejam ordenados.

Quick Sort

Considere o vetor:

[5, 3, 8, 2, 7]

- Escolhemos o último elemento como pivô:
7
- Separamos os elementos:
 - Menores que 7: [5, 3, 2]
 - Maiores que 7: [8]
- O pivô fica entre eles:

Quick Sort

- $[5, 3, 2] + [7] + [8]$
- Repetimos o processo para $[5, 3, 2]$:
 - Pivô: **2**
 - Menores: []
 - Maiores: $[5, 3]$
- Continuamos até que todas as partes estejam ordenadas.
- Resultado final:
- $[2, 3, 5, 7, 8]$

Quick Sort

- Utiliza a estratégia "**dividir para conquistar**".
- É um dos algoritmos de ordenação mais rápidos na prática.
- Não precisa comparar todos os elementos com todos os outros.
- Complexidade média: **$O(n \log n)$** .

Formas comuns de escolher o pivô

1. Primeiro elemento

Escolhe o primeiro elemento do vetor.

Exemplo:

[5, 3, 8, 2, 7]

↑

pivô = 5

Vantagem: simples de implementar.

Desvantagem: pode gerar desempenho ruim se o vetor já estiver ordenado

Formas comuns de escolher o pivô

2. Último elemento

- Escolhe o último elemento do vetor.

- Exemplo:

- [5, 3, 8, 2, 7]

↑

pivô = 7

- **Vantagem:** implementação simples e muito utilizada em exemplos didáticos.
- **Desvantagem:** também pode ser ineficiente em vetores já ordenados.

Formas comuns de escolher o pivô

3. Elemento central

- Escolhe o elemento do meio.
- Exemplo:
 - [5, 3, 8, 2, 7]
 ↑
 pivô = 8
- **Vantagem:** geralmente produz divisões mais equilibradas.

Formas comuns de escolher o pivô

4. Pivô aleatório

- Escolhe uma posição aleatória.
- Exemplo:
 - [5, 3, 8, 2, 7]
 ↑
 pivô = 3
- **Vantagem:** reduz a chance de cair no pior caso.
- É uma das estratégias mais utilizadas em implementações reais.

Formas comuns de escolher o pivô

5. Mediana de três (Median of Three)

- Escolhe a mediana entre:
- primeiro elemento,
- elemento central,
- último elemento.
- Exemplo:
- [5, 3, 8, 2, 7]

Primeiro = 5

Meio = 8

Último = 7

Mediana = 7

- **Vantagem:** costuma gerar partições mais equilibradas e melhorar o desempenho.

Qual é a melhor escolha do pivô?

Para fins didáticos:

- Último elemento ou primeiro elemento.

Para aplicações reais:

- Pivô aleatório.
- Mediana de três.

O objetivo é que o pivô fique o mais próximo possível do valor mediano do conjunto, pois isso divide o vetor em partes semelhantes e aproxima o desempenho ideal:

$O(n \log n)$

Quando o pivô é muito ruim (sempre o menor ou o maior elemento), o Quick Sort pode degradar para:

$O(n^2)$

Regra prática: quanto mais equilibradas forem as divisões geradas pelo pivô, mais rápido o Quick Sort será.

Quick Sort código:

```
// Função para trocar dois elementos
```

```
void trocar(int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Função de particionamento
```

```
int particionar(int vetor[], int inicio, int fim) {
```

```
    int pivo = vetor[fim]; // último elemento como pivô
```

```
    int i = inicio - 1;
```

```
    for (int j = inicio; j < fim; j++) {
```

```
        if (vetor[j] < pivo) {
```

```
            i++;
```

```
            trocar(&vetor[i], &vetor[j]);
```

```
        }
```

```
    }
```

```
    trocar(&vetor[i + 1], &vetor[fim]);
```

```
    return i + 1; // posição final do pivô
```

```
}
```

Quick Sort código:

```
// Função Quick Sort
void quickSort(int vetor[], int inicio, int fim) {
    if (inicio < fim) {
        int posicaoPivo = particionar(vetor, inicio, fim);
        quickSort(vetor, inicio, posicaoPivo - 1);
        quickSort(vetor, posicaoPivo + 1, fim);
    }
}
```

```
// Função para imprimir o vetor
void imprimirVetor(int vetor[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        printf("%d ", vetor[i]);
    }
    printf("\n");
}
```

Quick Sort código:

```
int main() {  
    int vetor[] = {5, 3, 8, 2, 7};  
    int tamanho = sizeof(vetor) / sizeof(vetor[0]);  
  
    printf("Vetor original:\n");  
    imprimirVetor(vetor, tamanho);  
  
    quickSort(vetor, 0, tamanho - 1);  
  
    printf("Vetor ordenado:\n");  
    imprimirVetor(vetor, tamanho);  
  
    return 0;  
}
```

Quick Sort código:

- Vetor inicial:

[5, 3, 8, 2, 7]

Pivô:

7

Após a partição:

[5, 3, 2, 7, 8]

↑

pivô

Quick Sort código:

O pivô **7** já está em sua posição definitiva.
Em seguida, o algoritmo ordena
recursivamente:

[5, 3, 2] e [8]

até obter:

[2, 3, 5, 7, 8]